

Virtual Queue Management SaaS

This document describes what was implemented in Phase 1: Supabase database architecture, migrations, Row Level Security (RLS), RPC foundations, and generated TypeScript database types.

Scope boundary: Phase 1 does NOT include dashboard UI, customer join UI, QR generation, **realtime subscriptions, notifications, payments, or analytics.**

Stack: Next.js, TypeScript, Supabase, PostgreSQL, Supabase Auth, RLS, RPCs.

1. Phase 1 goals

- ? Create a reproducible PostgreSQL schema via Supabase migrations.
- ? Enforce multi-tenant isolation with `business_id` and RLS.
- ? Provide secure customer RPCs without customer Auth accounts.
- ? Enforce queue entry state transitions in the database.
- ? Generate accurate TypeScript types from the live schema.
- ? Add typed Supabase clients (browser, server, admin).

2. Project files added or updated

2.1 Supabase

- ? `supabase/config.toml` ? local Supabase project configuration
- ? `supabase/seed.sql` ? empty non-production seed placeholder
- ? `supabase/migrations/*.sql` ? four ordered migration files (see section 3)

2.2 Application / library

- ? `lib/supabase/client.ts` ? browser client (anon key)
- ? `lib/supabase/server.ts` ? server client with cookies (anon key)
- ? `lib/supabase/admin.ts` ? server-only service-role client
- ? `lib/supabase/README.md` ? usage and secret-handling notes
- ? `types/database.ts` ? generated Database types
- ? `scripts/gen-db-types.mjs` ? regenerates types from local Supabase
- ? `.env.example` ? `NEXT_PUBLIC_SUPABASE_URL` / `ANON_KEY` (+ documented service role)
- ? `package.json` ? `db:*` scripts and `@supabase` dependencies
- ? `docs/architecture.md` ? section 16 Phase 1 architecture notes
- ? `README.md` ? Phase 1 setup instructions

2.3 Validation scripts

- ? `tests/integration/phase1_security_checks.sql` ? privilege invariants
- ? `tests/integration/phase1_behavior_checks.sql` ? isolation, transitions, numbering

3. Migration files (what each one does)

3.1 20260906180000_init_schema.sql

Creates the foundation schema.

- ? Enables `pgcrypto` (extensions schema) for secure token hashing.
- ? Creates enums: `member_role`, `queue_status`, `entry_status`.
- ? Creates tables: `profiles`, `businesses`, `business_members`, `services`, `queues`, `queue_entries`.
- ? Adds constraints (unique slug, positive service minutes, non-negative counters, valid statuses/roles).
- ? Adds indexes for membership, services, queues, and `queue_entries` query patterns.

entry/queue same business).

? Adds handle_new_user trigger (auth.users -> profiles).

? Adds handle_new_business trigger (creator becomes business_owner).

Design note: queue_entries.business_id is denormalized for efficient RLS without recursive joins.

3.2 20260906180100_rls_policies.sql

Implements tenant isolation and access control.

? Defines SECURITY DEFINER helpers: is_business_member(business_id), is_business_owner(business_id).

? Enables and forces RLS on every application table.

? profiles: users can select/update only their own row.

? businesses: members can read; authenticated can insert; owners can update/delete.

? business_members: peers can read; owners manage memberships.

? services / queues: members read; owners write.

? queue_entries: members may SELECT only; no direct INSERT/UPDATE/DELETE for anon or authenticated.

? Column grants hide access_token_hash from authenticated/anon clients.

3.3 20260906180200_rpc_functions.sql

Creates secure RPC foundations for customers and staff.

? join_queue(queue_id, customer_name, customer_phone?) ? validates open queue, locks queue row, increments current_number safely, stores SHA-256 token hash only, returns public_id + raw token + queue_number + status.

? get_ticket(public_id, access_token) ? verifies token hash; returns status, people_ahead, estimated_wait_minutes (people_ahead x average_service_minutes), service/business/queue names. Never returns token hash.

? cancel_ticket(public_id, access_token) ? waiting -> skipped only.

? transition_entry(entry_id, new_status) ? authenticated members only; enforces state machine.

Allowed transitions:

waiting -> called | skipped

called -> serving | skipped | no_show

serving -> completed | no_show

terminal -> completed | skipped | no_show

3.4 20260906180300_harden_function_grants.sql

Tightens EXECUTE privileges because Supabase may broadly grant execute on public functions.

? Revokes transition_entry and membership helpers from anon.

? Keeps join_queue / get_ticket / cancel_ticket available to anon + authenticated.

? Keeps internal hash/token helpers non-executable by clients.

4. Database objects summary

=====

4.1 Tables

? profiles ? id (FK auth.users), full_name, email, created_at

? businesses ? id, name, slug (unique), business_type, phone, email, logo_url, timestamps

? business_members ? business_id, user_id, role (business_owner|staff), unique(business_id,user_id)

? services ? business_id, name, description, average_service_minutes (>0), is_active

? queues ? business_id, service_id, name, status (open|paused|closed), current_number (>=0)

? queue_entries ? public_id, business_id, queue_id, customer fields, queue_number, status, access_token_hash, timestamps

4.2 Security model

- ? Customer access is token-based (raw token once; hash stored).
- ? Business A cannot read Business B data (membership + business_id RLS).
- ? Staff cannot directly mutate queue_entries; must use transition_entry.
- ? SUPABASE_SERVICE_ROLE_KEY must never be exposed to the browser.

5. How to apply and validate

=====

```
npx supabase start
npx supabase db reset    # or: npx supabase migration up --local
npm run db:types
npm run db:security-check
npm run db:behavior-check
npm run lint
npm run typecheck
npm run build
```

6. Environment variables

=====

- ? NEXT_PUBLIC_SUPABASE_URL ? public project URL
- ? NEXT_PUBLIC_SUPABASE_ANON_KEY ? public anon key (safe with RLS)
- ? SUPABASE_SERVICE_ROLE_KEY ? server only (lib/supabase/admin.ts)

7. Out of scope (later phases)

=====

- ? Phase 2: Auth + business provisioning UI
- ? Phase 3: Services/queues dashboard CRUD
- ? Phase 4: Public join + ticket pages
- ? Phase 5-6: Staff console + realtime
- ? Phase 7+: QR codes, team invites, hardening

Document generated for QueueLess Phase 1 ? database architecture and migrations.

Source of truth for SQL remains supabase/migrations/.